

6. RedisConf: replica

Redis 복제 시 **데이터 일관성**을 어떻게 확보할 수 있을까? 마스터에서 직접 읽거나 `WAIT` 명령어를 사용하여 복제본에 데이터가 도달했는지 확인하는 것이 중요합니다.

1. Redis 복제 기본 원리 및 필요성

Redis 복제는 데이터의 **고가용성 확보**, **읽기 부하 분산**, 그리고 **영속성 대체**를 위해 필수적이다.

1.1. Redis 복제 작동 방식

1. Redis 복제는 **마스터(Master)**와 **복제본(Replica)**으로 구성된다.
2. 클라이언트는 주로 마스터에게 데이터를 **쓰기(Write)** 요청하고, 이 데이터는 비동기적으로 복제본으로 **복제(Replicate)**된다.
3. 복제본은 여러 개를 가질 수 있으며, 클라이언트는 마스터 또는 복제본으로부터 데이터를 **읽기(Read)** 요청할 수 있다.
4. **페일오버(Failover)** 서비스(예: Sentinel)는 마스터와 복제본을 모니터링하며, 마스터 장애 시 복제본을 새로운 마스터로 승격시킨다.

1.2. Redis 복제의 핵심 특징

1. **비동기 복제**: 마스터에 데이터를 쓸 때 복제본에 도달하는 시점을 제어할 수 없다.
2. **마스터 비차단(Non-blocking)**: 복제 과정이 마스터의 클라이언트 응답 성능에 영향을 주지 않는다.
3. **복제본 읽기 가능**: 복제본에서 데이터를 읽을 수 있으나, 데이터 전송 중에는 응답이 지연될 수 있다.
4. **복제본 쓰기 제한**: 기본적으로 복제본은 읽기 전용이며, 복제본에 쓰기 요청 시 해당 쓰기는 로컬에만 적용되고 마스터 복제 시 덮어쓰여질 수 있다.
5. **계층적 복제**: 복제본이 다른 복제본의 마스터 역할을 수행하는 체인 구조도 가능하다.

1.3. Redis 복제가 필요한 이유

1. **고가용성(High Availability)**: 인메모리 데이터베이스인 Redis가 다운될 경우, 복제본이 즉시 데이터를 제공하여 서비스 중단을 방지한다.

2. **읽기 부하 분산(Spreading Read Load)**: 단일 Redis 인스턴스로 처리하기 어려운 대량의 읽기 요청을 여러 복제본으로 분산시켜 처리한다.
3. **영속성 대체(Replication in lieu of Persistence)**: 디스크 기반 영속성의 복잡성과 잠재적 장애 요인을 피하면서 데이터 복사본을 유지한다.

2. Redis 복제 방식: 전체 동기화와 부분 동기화

Redis 복제는 **전체 동기화(Full Sync)**와 **부분 동기화(Partial Sync)** 두 가지 방식으로 나뉜다.

2.1. 전체 동기화 (Full Sync)

1. **개념**: 마스터의 전체 데이터를 복제본으로 전송하는 방식이다.
2. **수행 방식**:
 - 마스터는 자체 프로세스를 **포크(Fork)**하여 별도의 프로세스를 생성한다.
 - 포크된 프로세스는 데이터를 디스크에 쓰거나, 디스크 없이 직접 네트워크를 통해 복제본으로 전송한다.
3. **사용 시점**:
 - 복제본이 처음 부트스트랩될 때, 즉 아무런 데이터가 없는 상태에서 사용된다.
 - 부분 동기화가 실패했을 경우에도 전체 동기화로 전환된다.

2.2. 부분 동기화 (Partial Sync)

1. **핵심 개념**: 복제 백로그(**Replication Backlog**)를 활용하여 변경된 데이터만 전송하는 방식이다.
2. **복제 백로그**:
 - 마스터는 일정 크기의 메모리 공간을 복제 백로그로 예약한다.
 - 새로운 쓰기 요청이 발생하면, 해당 데이터는 복제 백로그에도 저장된다.
 - 각 바이트마다 **오프셋(Offset)**이 증가하며 기록된다.
 - 복제 백로그는 **원형 버퍼(Circular Buffer)**로, 시작과 끝 오프셋을 가진다.
3. **복제 ID (Replication ID)**:
 - 마스터가 시작될 때 고유한 복제 ID가 부여된다.
 - 마스터가 재시작되면 새로운 복제 ID를 얻는다.

- 오프셋은 복제 ID와 함께 사용되어야 하며, 특정 복제 ID에 대한 특정 오프셋을 요청해야 한다.

4. 클라이언트 출력 버퍼 (Client Output Buffer):

- 네트워크 쓰기는 비용이 많이 들기 때문에, Redis는 클라이언트별 출력 버퍼를 사용하여 데이터를 모아 전송한다.
- 복제본 역시 클라이언트이므로 출력 버퍼를 가지며, 이는 부분 동기화에 영향을 준다.

5. 작동 과정:

- 클라이언트가 마스터에 데이터를 쓰면, 데이터는 키-값 저장소와 복제 백로그, 그리고 복제본의 출력 버퍼에 기록된다.
- 네트워크가 허용하는 속도로 데이터가 복제본으로 전송되며, 출력 버퍼에서 데이터를 소진시킨다.
- 복제 백로그는 나중에 연결될 복제본들을 위해 데이터를 보관한다.

6. 복제 백로그 관련 설정: `repl_backlog_size` 로 크기를 조절할 수 있다.

7. 오프셋:

- **첫 바이트 오프셋(First Byte Offset):** 원형 버퍼가 가득 차 덮어쓰기 시작할 때 증가하며, 유효한 데이터 범위를 나타낸다.
- **마스터 복제본 오프셋(Master Replica Offset):** 쓰기 작업마다 증가하는 전체 오프셋이다.
- 유효한 데이터는 첫 바이트 오프셋과 마스터 복제본 오프셋 사이에 존재한다.

3. Redis 복제 설정 및 모니터링

Redis 복제를 설정하고 상태를 확인하기 위한 다양한 명령어가 존재한다.

3.1. 복제본 설정 방법

Basic Configuration Options

- replicaof <master-ip> <master-port>
 - replicaof localhost 7001
 - replicaof NO ONE
- repl-backlog-size 1mb
- repl-timeout 60
- client-output-buffer-limit replica 256mb 64mb 60

1. 복제본으로 설정: `REPLICAOF <host> <port>` 명령어를 사용하여 특정 마스터의 복제본으로 설정한다.
2. 마스터로 전환: `REPLICAOF NO ONE` 명령어를 사용하여 복제본 역할을 중단하고 마스터가 된다.

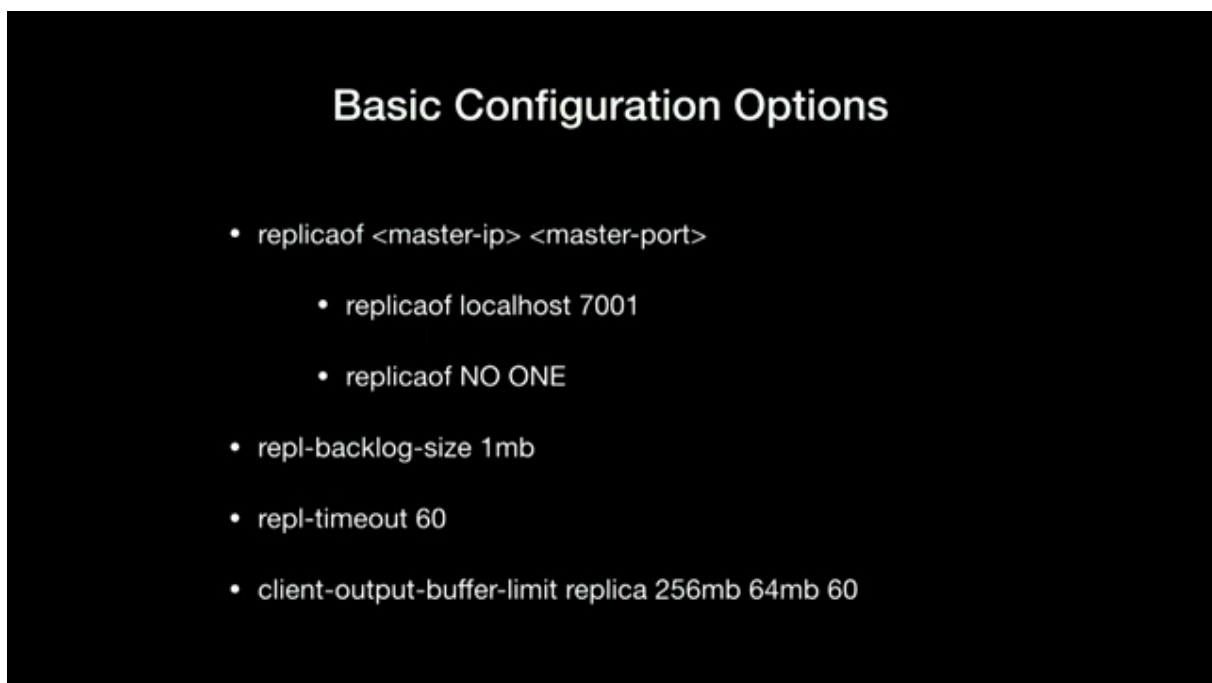
3.2. 주요 설정 옵션

Basic Configuration Options

- replicaof <master-ip> <master-port>
 - replicaof localhost 7001
 - replicaof NO ONE
- repl-backlog-size 1mb
- repl-timeout 60
- client-output-buffer-limit replica 256mb 64mb 60

1. `repl_backlog_size`: 복제 백로그의 크기를 설정하며, 기본값은 1MB이다.
2. `replica_timeout`: 마스터와 복제본 간 데이터 전송이 완료되어야 하는 최대 시간(기본 60초)을 설정한다. 이 시간을 초과하면 연결이 끊긴다.
3. `client-output-buffer-limit`: 복제본의 출력 버퍼 제한을 설정한다.
 - **하드 리밋(Hard Limit)**: 기본 256MB이며, 초과 시 즉시 연결이 종료된다.
 - **소프트 리밋(Soft Limit)**: 기본 64MB/60초이며, 이 임계값을 초과하면 연결이 종료된다.

3.3. 복제 상태 모니터링



1. `ROLE` 명령어:
 - 마스터에서 실행 시: 자신의 역할(master), 오프셋, 연결된 복제본 정보(IP, 포트, 오프셋)를 보여준다.
 - 복제본에서 실행 시: 자신의 역할(replica), 마스터 정보(IP, 포트, 상태), 자신의 오프셋을 보여준다.
2. `INFO REPLICATION` 명령어:
 - **마스터 정보**: 연결된 복제본 수, 각 복제본의 IP, 포트, 상태(connected, online), 오프셋, 그리고 `lag` (마지막 통신 시간)을 보여준다.

- `lag`: 일반적으로 0~1초 사이이며, 값이 증가하면 복제본 연결 또는 데이터 전송에 문제가 있음을 나타낸다.
- **복제 ID, 백로그 관련 정보**
보: `replid`, `repl_backlog_first_byte_offset`, `repl_backlog_size` 등을 확인할 수 있다.
- **복제본 정보**: 복제본의 오프셋, 마스터 정보, 복제 ID, 첫 바이트 오프셋, 백로그 크기 등을 보여준다.

3. 두 번째 복제 ID (Second Replica ID):

- 마스터가 장애 복구 후 새로운 마스터가 되었을 때, 이전 마스터의 복제 ID를 저장하여 기존 복제본들이 원활하게 재연결되도록 돕는다.
- 이는 대규모 복제 환경에서 전체 동기화로 인한 비효율을 줄여준다.

4. Redis 복제 시 데이터 일관성 확보 방안

Redis는 기본적으로 100% 일관성을 보장하지 않지만, 몇 가지 방법을 통해 **최선 노력 일관성(Best Effort Consistency)**을 확보할 수 있다.

4.1. 데이터 불일치 발생 시나리오

1. 클라이언트가 마스터에 데이터를 설정한 직후, 복제본에 아직 복제되지 않은 상태에서 복제본으로 읽기 요청을 하면, 이전 값을 받거나 키를 찾지 못하는 문제가 발생한다.
2. Redis는 분산 시스템이므로 데이터 복제 시점은 예측하기 어렵다.

4.2. 데이터 일관성 확보 전략

Best Effort Consistency

- Read only from Master
- Server config: min-replicas-to-write, min-replicas-max-lag
- Client call: wait num-replicas time-out-millis
- Client slows write rate based on the replication offset difference between Master and Replica

1. 마스터에서 읽기:

- 가장 간단한 방법으로, 데이터가 항상 최신 상태인 마스터에서 직접 읽는다.
- 하지만 마스터 장애 시 복제본으로 전환될 때 마지막 쓰기 작업이 누락될 가능성이 있어 100% 보장은 어렵다.
- 복원 작업 등 마스터 읽기가 불가능한 경우에만 복제본을 사용한다.

2. MIN REPLICAS 설정:

- min-replicas-to-write 와 min-replicas-max-lag 설정을 통해 쓰기 작업을 허용할 복제본의 최소 개수와 최대 지연 시간을 지정한다.
- 이 설정은 쓰기 작업이 충분히 복제될 수 있는 상태일 때만 쓰기를 허용하여 일관성 창을 줄인다.
- 하지만 이 설정 역시 데이터가 해당 복제본에 도달했음을 보장하지는 않는다.

3. WAIT 명령어 사용:

- 클라이언트가 데이터를 쓴 후, 지정된 수의 복제본에 데이터가 도달할 때까지 기다리도록 요청할 수 있다.
- WAIT <num_replicas> <timeout> 형식으로 사용하며, 타임아웃 설정이 필수적이다.
- 주의: WAIT 명령어는 데이터를 쓴 동일한 연결(Connection)에서 실행해야 정확한 결과를 얻을 수 있다. 연결 풀링(Connection Pooling) 사용 시 주의가 필요하다.

4. 협력적 로더 클라이언트(Cooperative Loader Client):

- 대량의 데이터를 로드하는 클라이언트가 주기적으로 마스터와 복제본의 복제 오프셋을 비교하여 동기화 상태를 확인하고, 동기화가 충분히 이루어졌을 때 다음 데이터 배치를 로드하는 방식이다.

5. Redis 복제 시 메모리 사용량 관리

Redis 복제 환경에서 메모리 사용량은 키-값 저장소, 복제 백로그, 그리고 클라이언트 출력 버퍼에 의해 결정된다.

5.1. 메모리 할당 및 사용처

1. 주요 메모리 사용처:

- **키-값 저장소**: Redis의 핵심 기능으로 데이터를 저장하는 데 사용된다.
- **복제 백로그 버퍼**: 부분 동기화를 위해 변경된 데이터를 저장한다.

2. `maxmemory` 설정: Redis 전체 메모리 사용량을 제한하는 설정이다.

3. 메모리 분할: `maxmemory` 는 키-값 저장소, 복제 백로그 버퍼, 그리고 클라이언트 출력 버퍼에 의해 나누어 사용된다.

4. 클라이언트 출력 버퍼의 특징:

- 각 복제본은 별도의 출력 버퍼를 가지며, 이 버퍼의 크기는 `maxmemory` 설정에 포함되지 않는다.
- 따라서 실제 프로세스 메모리 사용량은 `maxmemory` 보다 더 커질 수 있다.

5. 복제 백로그: 모든 복제본이 공유하는 단일 원형 버퍼이며, 사용자 키를 위한 메모리를 감소시킨다.

6. 클라이언트 출력 버퍼: 복제본마다 존재하며, 여러 복제본이 있을 경우 총 메모리 사용량이 크게 증가할 수 있다 (기본 256MB).

5.2. 일반적인 메모리 관련 문제 및 해결 방안

Memory Usage Key Points

- Replication Backlog
 - Single circular buffer shared by all replicas
 - Reduces memory available for user keys
- Replica Client Output Buffer
 - One buffer per replica
 - Buffer has hard and soft limits
 - Increases memory used by process

1. 전체 동기화 실패 (Fork Error):

- **원인:** 마스터가 전체 동기화를 위해 프로세스를 포크할 때 메모리가 부족하여 발생하는 오류이다.
- **해결 방안:**
 - 포크 시 필요한 메모리 양을 고려하여 마스터와 복제본 간의 적절한 메모리 여유 공간을 확보한다.
 - `vm.overcommit_memory` 커널 설정을 1로 변경하여 OS가 메모리 할당을 더 유연하게 처리하도록 한다. (Redis 시작 시 권장 메시지 출력)

2. 복제본 타임아웃 (Replica Timeout):

- **원인:** 데이터셋이 너무 크거나 네트워크 연결이 불안정하여 전체 동기화 데이터 전송이 `replica_timeout` 내에 완료되지 못하는 경우 발생한다.
- **해결 방안:** `replica_timeout` 값을 늘리거나, 전체 동기화 데이터 크기를 줄인다.

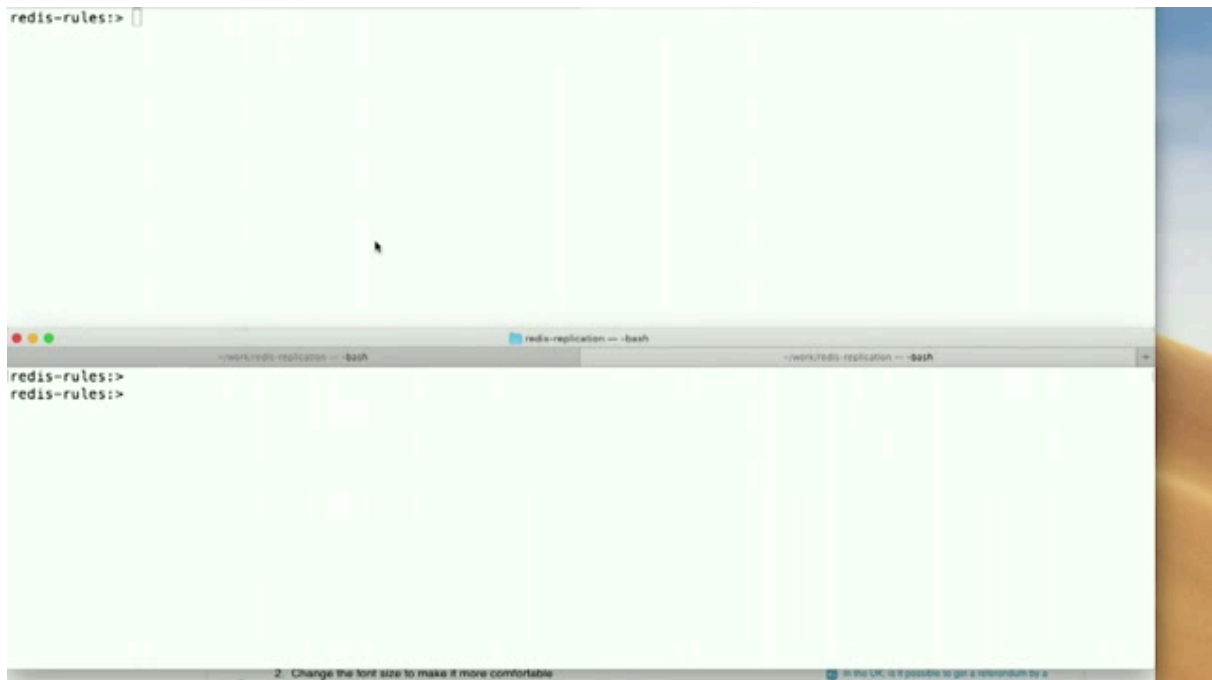
3. 부분 동기화 실패 (Client Scheduled to be Closed ASAP):

- **원인:** 복제본의 출력 버퍼가 과부하되어 마스터가 더 이상 데이터를 버퍼링하지 않고 연결을 종료하는 경우 발생한다.
- **해결 방안:** `client-output-buffer-limit` 설정을 늘린다. (단, 버퍼는 무한하지 않으므로 비용 고려 필요)

6. 데모 시연: 복제 동작 및 문제 해결

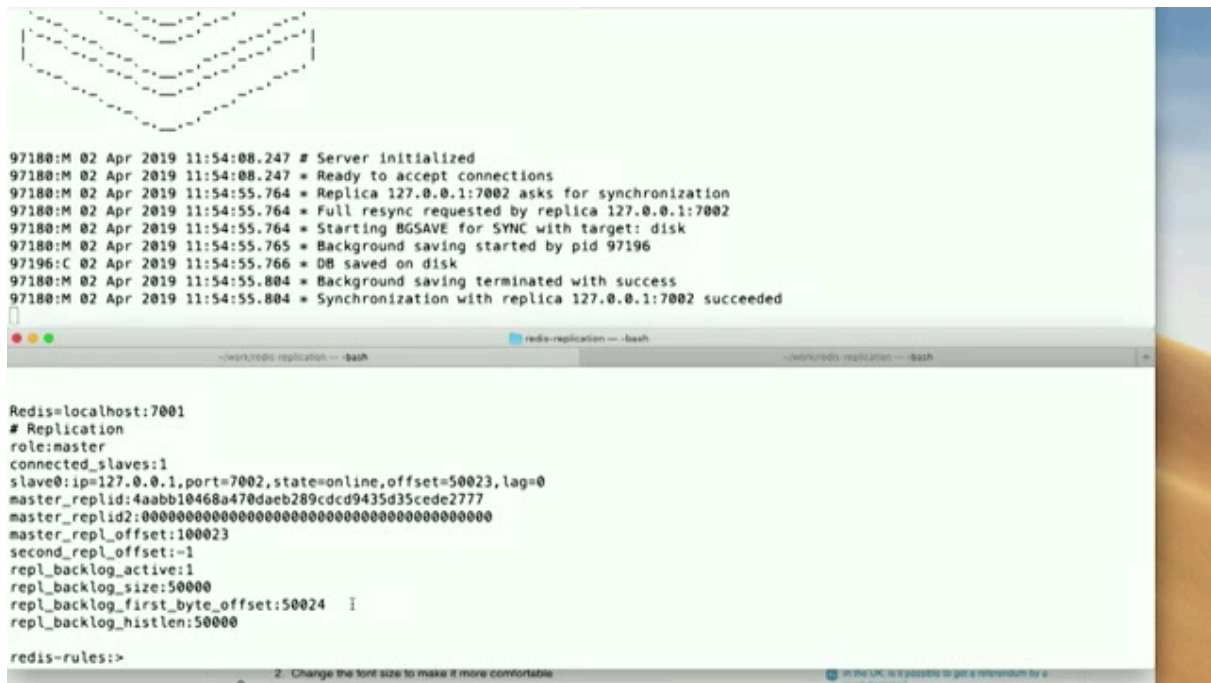
실제 환경에서 Redis 복제가 어떻게 작동하는지, 그리고 발생할 수 있는 문제들을 어떻게 해결하는지 시연을 통해 보여준다.

6.1. 복제 기본 설정 및 모니터링 시연



1. 마스터(7001 포트)와 복제본(7002 포트)을 시작하고 `ROLE` 및 `INFO REPLICATION` 명령어로 상태를 확인한다.
2. 마스터에 데이터를 쓰면서 마스터와 복제본의 오프셋이 동기화되는 과정을 관찰한다.
3. 복제 백로그 크기(`repl_backlog_size`)에 따라 첫 바이트 오프셋(`first_byte_offset`)이 어떻게 변하는지 확인한다.

6.2. 복제 불안정성 시뮬레이션 및 복구



[illegible]

- #### 6.4. 리더십 전환과 두 번째 복제 ID의 영향 시연



The screenshot shows a terminal window with a macOS-style title bar. The terminal is running a Redis configuration script. The output shows the configuration of Redis rules and the setup of Redis replication. The terminal text is as follows:

```
redis-rules:> rc1
redis-cli -p 7001
127.0.0.1:7001> set k1 v1
OK
127.0.0.1:7001> wait 1 10
(integer) 1
127.0.0.1:7001> set k2 v2
OK
127.0.0.1:7001> wait 1 10
(integer) 0
127.0.0.1:7001> wait 1 10
(integer) 0
127.0.0.1:7001> wait 1 10
(integer) 1
127.0.0.1:7001> quit
redis-rules:>
```

1. 마스터십을 7001에서 7002로 전환하는 과정을 시연한다.
2. 리더십 전환 후, 새로운 마스터는 이전 마스터의 복제 ID와 오프셋을 두 번째 복제 ID 및 오프셋으로 저장하여 기존 복제본들이 원활하게 재연결되도록 돕는 것을 확인한다.

6.5. 메모리 사용량 및 버퍼 영향 시연



1. `maxmemory` 설정 하에서 복제 백로그 크기가 증가함에 따라 사용자 키를 위한 메모리가 줄어들어 저장 가능한 키의 수가 감소하는 것을 보여준다.
2. `client-output-buffer-limit` 설정을 기본값(256MB)으로 설정하고 복제본을 중지시키면, 출력 버퍼가 증가했다가 연결 종료 후 감소하는 것을 관찰한다.
3. `client-output-buffer-limit` 을 0으로 설정하고 복제본을 중지시키면, 메모리 사용량이 무제한으로 증가하는 위험성을 보여준다. (실제 환경에서는 절대 사용 금지)